

T-DT 2027 青训计划 学习任务(2)

C++面向对象编程小练习

学习目标

这次练习不只是“把面积算出来”，重点是通过一个小型图形管理程序理解 C++ 面向对象编程中最常用的几个概念：

1. **封装**：把图形自身的数据和操作放进类中，不要把半径、边长等数据全部暴露在 `main()` 里处理。
2. **继承**：用统一的 `Shape` 基类描述“图形”，再由 `Circle`、`Rectangle`、`Triangle` 派生出具体图形。
3. **多态**：主程序只通过 `Shape*` / `Shape&` 调用统一接口，不需要知道当前对象究竟是哪一种图形。
4. **构造与析构**：理解对象如何初始化，以及为什么通过基类指针管理派生类对象时需要虚析构函数。
5. **职责划分**：尽量让每个类只负责自己的数据和行为，不要把所有逻辑都堆在 `main()` 中。

题目：图形管理器

编写一个 C++ 程序，用于管理不同类型的图形。定义抽象基类 `Shape`，以及派生类 `Circle`、`Rectangle` 和 `Triangle`。

程序首先完成基础的面积计算，在此基础上继续增加周长、统计和缩放等功能。重点不是使用大量 `if-else` 完成所有功能，而是让不同对象通过统一接口表现出各自的行为。

基础要求

1. 定义抽象基类 `Shape`，至少包含纯虚函数 `area()`，用于计算图形面积。
2. 派生类 `Circle`、`Rectangle`、`Triangle` 分别实现自己的 `area()`。

3. 图形所需的数据应保存在对应类的成员变量中，并通过构造函数完成初始化。
4. 使用基类指针或引用保存、访问不同类型的派生类对象，并通过多态计算面积。
5. 不允许在输出面积时重新通过图形类型写三套面积计算逻辑；面积计算应由对象自身完成。
6. 输出面积统一保留两位小数。

输入

第一行输入一个整数 n ，表示有 n 个图形。

接下来的 n 行，每行第一个单词表示图形类型，后面跟该图形所需的参数：

- Circle r : 半径为 r 。
- Rectangle w h : 长、宽分别为 w 、 h 。
- Triangle a b c : 三条边长分别为 a 、 b 、 c 。

基础输出

输出 n 行，每行表示对应图形的面积，保留两位小数。

输入样例 1

```
4
Circle 3
Rectangle 2 6
Triangle 6 8 10
Circle 7
```

输出样例 1

```
28.27
12.00
24.00
153.94
```

输入样例 2

```
5
Rectangle 1 1
Rectangle 3 4
Circle 2
Triangle 5 12 13
Circle 4
```

输出样例 2

```
1.00
12.00
12.57
30.00
50.27
```

功能练习

完成基础要求后，在原有类结构上继续增加下面的功能。不要为每一种图形在 `main()` 中单独写一套实现。

1. **周长接口**：在 `Shape` 中增加纯虚函数 `perimeter()`，三个派生类分别实现自己的周长计算。
2. **类型接口**：增加 `type()` 或类似接口，使程序可以通过基类指针输出对象的类型，例如 `Circle`、`Rectangle`、`Triangle`。
3. **统一信息输出**：输出每个对象的类型、面积和周长，例如：`Circle area=28.27 perimeter=18.85`。可以自己设计排版，不要求与示例完全一致。
4. **统计功能**：基于 `vector<Shape*>` 或其他容器，计算所有图形的总面积，并找出面积最大的图形。统计逻辑只能依赖 `Shape` 的公共接口。
5. **缩放功能**：在 `Shape` 中增加 `scale(double factor)` 接口。调用 `shape->scale(2.0)` 后，圆的半径、矩形的长宽、三角形的三条边都应按比例放大，而主程序不需要判断对象具体是哪一种图形。
6. **虚析构函数**：为 `Shape` 添加虚析构函数，并在程序结束前正确释放通过 `new` 创建的对象。思考：如果基类析构函数不是 `virtual`，通过 `Shape*` 删除派生类对象可能产生什么问题？

进一步练习（加分项）

下面内容不要求全部完成，可以根据自己的进度选择：

1. 使用 `std::sort` 按面积从小到大排序，但排序代码仍然只调用 `Shape::area()`。
2. 对输入参数做合法性检查，例如半径必须大于 0，三角形必须满足两边之和大于第三边。
3. 将 `vector<Shape*>` 改写为 `vector<unique_ptr<Shape>>`，了解智能指针与 RAII，做到不手动 `delete`。
4. 把“根据字符串创建不同图形对象”的逻辑从 `main()` 中拆出去，例如写成 `createShape()` 函数；思考这样做对代码可读性和后续扩展有什么好处。
5. 新增一种图形（例如 `Square` 或 `Ellipse`）。如果新增图形时需要大改原来的统计、排序、缩放代码，说明你的多态设计仍有改进空间。

编程要求

1. 建议使用 C++17 或更高版本。
2. 类的数据成员尽量设为 `private`，通过构造函数和成员函数维护对象状态。
3. 能使用 `const` 的成员函数尽量写成 `const`，例如 `double area() const`。
4. 派生类重写虚函数时使用 `override`，帮助编译器检查函数签名。
5. 不建议使用非标准的变长数组 `Shape* shapes[n]`，推荐使用 `std::vector`。
6. 编译时建议打开常用警告：

```
g++ -std=c++17 -Wall -Wextra -pedantic main.cpp -o oop_shape
./oop_shape
```

提交内容

1. 提交完整源码；如果拆分为多个 `.h/.cpp` 文件，则提交完整工程。
2. README 中说明使用了哪些面向对象特性，并附基础任务运行结果；功能练习或加分项可继续附上结果和简要思路。